

Agenda

- Event Handling
- Props
- State
- Hooks
- Hook-useState
- React Router
- axios and toastify

Component Event Handling

- Components should handle its events to make the component easily reusable.
- Event handling in React follows camelCase naming convention (`onClick` instead of `onclick`)
- Uses Synthetic Events - React's cross-browser wrapper around native DOM events
- Event handlers receive a `SyntheticEvent` object containing event details

Basic Event Handling

- define a event handler inside a component
- event handler
 - function which handles a required event
 - e.g. `onButtonClick()` => click event of a button
- use event names provided by react
 - e.g. `click` => `onClick`

```
function App() {  
  // 1. Define handler function  
  const handleButtonClick = () => {  
    alert("Button clicked!");  
  };  
  
  return (  
    <div>  
      { /* 2. Pass function reference (no parentheses!) */ }  
      <button onClick={handleButtonClick}>Click Me</button>  
    </div>  
  );  
}
```

- Pass function reference - `onClick={handleClick}`
- Avoid function call - `onClick={handleClick()}` would execute immediately
- `SyntheticEvent` provides consistent interface across browsers

Handle user inputs

```
function App() {
  const handleInputChange = (event) => {
    // Access input value
    const userInput = event.target.value;
    console.log(`User input: ${userInput}`); // Correct string interpolation
  };
  return (
    <div>
      <label>Username:</label>
      <input
        type="text"
        onChange={handleInputChange}
        placeholder="Type something..."
      />
    </div>
  );
}
```

- Use `event.target.value` for input elements
- Always use proper string interpolation with backticks: `` `Value: \${value}` ``

Form submission Example

```
function FormComponent() {
  const handleSubmit = (event) => {
    event.preventDefault();
    console.log("Form submitted!");
  };
  return (
    <form onSubmit={handleSubmit}>
      <input type="text" onChange={(e) => console.log(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}
```

Common Event Types

Event	React Prop	DOM Equivalent
Click	<code>onClick</code>	<code>click</code>
Input Change	<code>onChange</code>	<code>input/change</code>
Form Submission	<code>onSubmit</code>	<code>submit</code>
Mouse Enter	<code>onMouseEnter</code>	<code>mouseenter</code>
Key Press	<code>onKeyDown</code>	<code>keydown</code>

Event Handler Best Practices

1. Naming Convention : Use `handle[Element][Event]` pattern

```
handleButtonClick, handleInputChange;
```

2. Inline Handlers (for simple logic):

```
<button onClick={() => console.log("Clicked")}>Log Click</button>
```

3. Parameter Passing:

```
<button onClick={(e) => handleClick(id, e)}>Delete Item</button>
```

SyntheticEvent Details

- Properties:

```
event.target; // DOM element that triggered event
event.currentTarget; // Element where handler is attached
event.type; // Event type ('click', 'change', etc.)
```

- Methods:

```
event.preventDefault(); // Cancel default behavior
event.stopPropagation(); // Stop event bubbling
```

Props (Parent->Child Data Flow)

- It is an object which will be passed from a parent to child component
- It is a parameter to the component function
- props are readonly i.e the child must not modify the data sent by the parent component (even if the child modifies the props, the parent won't be able to see the changes)

```
// child
function Person(props) {
  return (
    <div>
      <h3>Person</h3>

      {/* reading the data sent by the parent component */}
    </div>
  );
}
```

```

        <div>name = {props.name}</div>
      </div>
    )
  }

// child
// props gets destructured into model and company variables
function Car({ model, company }) {
  return (
    <div>
      <h3>Car</h3>

      {/* reading the data sent by the parent component */}
      <div>model = {model}</div>
      <div>company = {company}</div>
    </div>
  )
}

// parent
function App() {
  const name = 'person1'
  const car = { model: 'triber', company: 'renault' }
  return (
    <div>
      <h2>Welcome to react</h2>
      {/* the parent is passing the data to the child component */}
      <Person name={name} />
      <Car
        model={car['model']}
        company={car['company']}
      />
    </div>
  )
}

```

Single Responsibility Principle

- when data is passed from parent component to child component using props it is recommended for the components to follow single responsibility principle
- In this principle it is recommended to pass the properties of object individually rather than passing the entire object.
- Passing a large object can lead to components having access to data they don't need, potentially encouraging them to take on responsibilities outside their intended scope.
- Increased Coupling: Passing entire objects can create tighter coupling between parent and child components. If the structure of the object changes, many child components might need to be updated, even if they only use a small part of the object. Passing only necessary properties reduces this coupling.

```

// Good: Focused component
function BookPrice({ price }) {

```

```
    return <div>Price: Rs. {price}</div>;  
  }  
  
  // Bad: Mixed responsibilities  
  function BookDetails({ book }) {  
    // Handles price, image, author...  
  }  
}
```

React Hook

- a JS function defined by react which is used to perform some logic on the components
- These are special functions prefixed with **use** that let you "hook into" React features
- react introduced the react hooks in version 16.0
- e.g.
 - built-in hooks
 - useState()
 - useContext()
 - useEffect()
 - useReducer()
 - useCallback()
 - useMemo()
 - useRef()
 - useId()
 - react router hooks
 - useNavigate()
 - useParams()
 - useLocation()
 - redux hooks
 - useDispatch()
 - useSelector()

Rules to call React Hooks

- Never call inside loops, conditions, or nested functions
- Ensures consistent hook order between renders
- Only Call Hooks from React Functions like
 1. Functional components
 2. Other custom hooks

1. State Management Hooks

Hook	Purpose	Key Characteristics
useState()	Creates component state	Returns <code>[state, setState]</code> pair
useReducer()	Manages complex state logic	Alternative to useState for complex state objects
useContext()	Accesses shared context	Avoids prop drilling through component tree

2. Performance Optimization Hooks

Hook	Purpose	Key Characteristics
<code>useMemo()</code>	Memoizes expensive computations	Caches calculated values between renders
<code>useCallback()</code>	Memoizes callback functions	Prevents unnecessary re-renders of child components

3. Side Effect & DOM Hooks

Hook	Purpose	Key Characteristics
<code>useEffect()</code>	Handles side effects	Replaces lifecycle methods (<code>componentDidMount</code> , <code>componentDidUpdate</code>)
<code>useRef()</code>	Accesses DOM elements directly	Creates mutable object that persists across renders

4. React Router Hooks

Hook	Purpose	Usage Example
<code>useNavigate()</code>	Programmatic navigation	<code>const navigate = useNavigate(); navigate('/home');</code>
<code>useLocation()</code>	Accesses current route info	<code>const location = useLocation(); location.pathname</code>
<code>useParams()</code>	Reads route parameters	<code>const { id } = useParams();</code>

5. Redux Toolkit Hooks

Hook	Purpose	Key Characteristics
<code>useDispatch()</code>	Gets Redux store dispatcher	<code>const dispatch = useDispatch(); dispatch(action());</code>
<code>useSelector()</code>	Reads from Redux store	<code>const user = useSelector(state => state.auth.user);</code>

When to Use Each Hook

Scenario	Recommended Hook
Form input state	<code>useState</code>
Global theme settings	<code>useContext</code>
API data fetching	<code>useEffect</code> + <code>useState</code>
Complex state logic	<code>useReducer</code>

Scenario	Recommended Hook
Optimizing expensive calculations	<code>useMemo</code>
Preventing child re-renders	<code>useCallback</code>
Managing focus/animations	<code>useRef</code>
Routing navigation	<code>useNavigate</code>
Redux state management	<code>useSelector</code> + <code>useDispatch</code>

State

- object which represents the component's inner data
- you may use one or more state members
- when there is a change in the state (members), the component rerenders the application with updated values
- state is a read writable object
- to create a state inside the component
 - inside functional component: `useState()`
 - inside class component: class components already has gotten a property (state) with all the properties and their values
- every component creates its own state
- unlike the props, the state is not meant for sharing the data with the child

Q. Why local variables are not standard practice in React

1. They don't survive re-renders
 - Every time a component re-renders, its function body runs again → local variables are re-initialized.
2. React won't track changes
 - React's UI updates are tied to state. If you update a local variable,
 - React has no idea something changed, so:
 - Controlled inputs don't update properly
 - Validation logic depending on values won't re-run
 - Conditional rendering may fail
3. Breaks the "React way"
 - React encourages unidirectional data flow and state-driven UI.
 - Local variables bypass this, making your code unpredictable and harder to debug.

`useState()` Hook

- Adds reactive state to functional components
- It returns an array with two members
- Returns a stateful value at `arr[0]` and a function at `arr[1]` to update it
- It is used to maintain the component's state

- when the state get updated, the component gets re-rendered
- accepts only one parameter to initialize the state member
- first member array[0]: reference to the value inside the state (getter)
- second member array[1]: reference to the function used to update the value (setter)

- Syntax

```
const [state, setState] = useState(initialValue);
```

Part	Purpose
state	Current state value (read-only)
setState	Updater function (triggers re-render)
initialValue	Initial state (can be any type - number, string, object, etc.)

- Example

```
function Counter() {  
  const [counter, setCounter] = useState(0)  
  
  const onIncrement = () => {  
    // trigger the component rerendering  
    setCounter(counter + 1)  
  }  
  
  const onDecrement = () => {  
    // trigger the component rerendering  
    setCounter(counter - 1)  
  }  
  
  return (  
    <div>  
      <h3>Counter</h3>  
  
      <div>counter = {counter}</div>  
      <button onClick={onIncrement}>Increment</button>  
      <button onClick={onDecrement}>Decrement</button>  
    </div>  
  )  
}  
export default Counter
```

component rerender

- invoking the component with required features with updated value
- to trigger rerendering update the component state

Third party libraries

1. react router
2. axios
3. toastify

1. React Router

- It is used for component navigation
- used to add routing feature in react application
- routing feature will enable the react application to switch between the components dynamically
- provides router which does the routing
- to install the react router use the below command

```
# installation
> npm install react-router-dom
> yarn add react-router-dom
```

- To use it we need to put our main component inside
- wrap the App component inside the BrowserRouter

```
// main.jsx

import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import { BrowserRouter } from 'react-router-dom'

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
)
```

- define all the routes
- route
 - mapping between
 - path
 - element (component)

```
// App.jsx

import React from 'react'
import Login from './pages/Login/Login'
import Register from './pages/Register/Register'
import Home from './pages/Home/Home'
```

```
import Cart from './pages/Cart/Cart'
import Orders from './pages/Orders/Orders'
import { Route, Routes } from 'react-router-dom'

function App() {
  return (
    <div>
      <Routes>
        <Route
          path='/login'
          element={<Login />}
        />
        <Route
          path='/register'
          element={<Register />}
        />
        <Route
          path='/cart'
          element={<Cart />}
        />
        <Route
          path='/home'
          element={<Home />}
        />
        <Route
          path='/orders'
          element={<Orders />}
        />
      </Routes>
    </div>
  )
}

export default App
```

- BrowserRouter
 - used to add routing feature for website (browser)
 - must be the parent of in order to add the routing feature
- Routes
 - collection of Route objects
 - contains all the possible routes of an application
- Route
 - mapping between path (to launch the component) and element (component to be launched)
- There are two ways to navigate to another component

1. static navigation

- the navigation performed without executing any JS code (performed using JSX)
- no condition is required to launch the new component
- can be performed by using `<Link></Link>`

```
function Login() {  
  return (  
    <div>  
      <h2>Login</h2>  
      <Link to='/register'>Register here</Link>  
    </div>  
  )  
}
```

2. dynamic navigation

- navigation is performed executing the JS code
- also known as conditional navigation
- can be achieved by a react hook named `useNavigate()`

```
import { useNavigate } from 'react-router-dom'  
  
function Register() {  
  // get the reference of navigate function  
  const navigate = useNavigate()  
  
  const onRegister = () => {  
    // navigate to login page  
    navigate('/login')  
  }  
  
  return (  
    <div>  
      <h2>Register</h2>  
      <button onClick={onRegister}>Login</button>  
    </div>  
  )  
}  
  
export default Register
```

2. axios

- used to call the REST APIs
- alternative to JS fetch API

```
# installation  
> npm install axios
```

```
> yarn add axios
```

```
import axios from 'axios'

async function getProducts() {
  try {
    const url = 'http://localhost:4000/products'
    const result = await axios.get(url)
    console.log(result)
  } catch (ex) {
    console.log(ex)
  }
}

async function createProduct(title, price, description) {
  try {
    const url = 'http://localhost:4000/products'
    const body = { title, price, description }
    const result = await axios.post(url, body)
    console.log(result)
  } catch (ex) {
    console.log(ex)
  }
}
```

3. react toastify

- used to show the toast message

```
# installation
> npm install react-toastify
> yarn add react-toastify
```

```
import { ToastContainer } from 'react-toastify'

function App() {
  return (
    <div>
      {/* add the toast container to show the toast messages */}
      <ToastContainer />
    </div>
  )
}

export default App
```

```
import { toast } from 'react-toastify'

function Dummy() {
  const onShow = () => {
    toast.success('this is a success message')
  }

  return (
    <div>
      <button onClick={onShow}>show success</button>
    </div>
  )
}
```

SUNBEAM INFOTECH